

PokerCRR

Applying Cox-Ross-Rubinstein binomial pricing logic to poker decision-making

This project treats a poker hand as a discrete-time stochastic process, the same structure a CRR binomial tree uses to price an option: equity plays the role of the underlying's price, streets play the role of time steps, and betting decisions are priced using the same no-arbitrage logic used for options and other derivatives. Four modules implement this end to end: Monte Carlo equity estimation, martingale-based bet pricing, a Kuhn Poker GTO solver trained via Counterfactual Regret Minimization, and Kelly Criterion bankroll sizing.

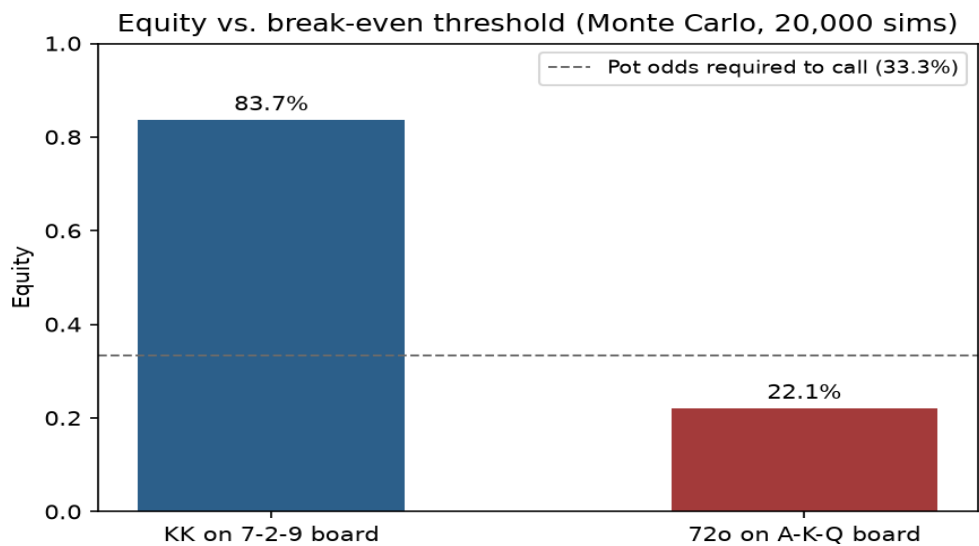
Methodology: the CRR-to-poker mapping

The Cox-Ross-Rubinstein (CRR) model prices an option by building a binomial tree: at each discrete time step, the underlying's price moves up or down, and the option's value is computed by working backward from the known payoffs at expiry, applying risk-neutral probabilities at every node. A poker hand has the same discrete-time structure: it moves through a fixed sequence of steps (preflop, flop, turn, river), and at each step new public information (a card) shifts the probability of winning, exactly as a price move shifts an option's moneyiness.

Options / CRR concept	Poker analogue	Implemented in
Underlying's price	Equity (win probability)	equity/monte_carlo.py
Risk-neutral pricing	Pot odds / EV of a call	pricing/martingale.py
Backward induction	Solving for equilibrium play	gto/kuhn_solver.py
Position sizing under a model	Bankroll sizing given an edge	bankroll/kelly.py

1. Equity Estimation and Pricing

The equity module runs Monte Carlo simulation (20,000 random runouts per hand) to estimate win probability at any point in a hand. The pricing module then applies pot-odds logic, the poker equivalent of risk-neutral pricing, to decide whether a call is +EV. Below is a broader set of six hands spanning strong made hands, draws, and air, all facing the same bet size (pot=100, facing a 50-chip bet).



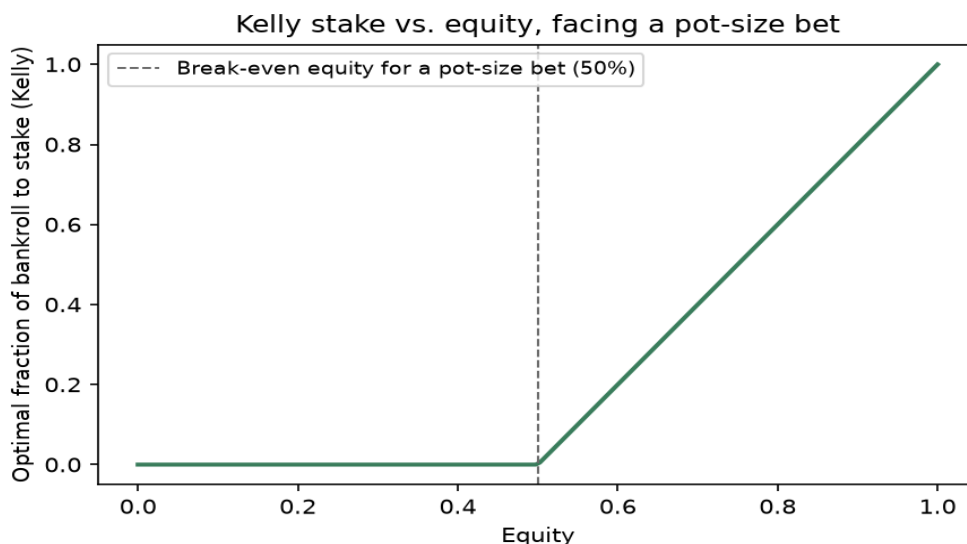
Two representative cases from the table below, shown visually: a strong overpair clears the pot-odds threshold comfortably, while pure air does not.

Hand	Equity	EV of call	Decision	Full Kelly
Overpair (AA)	85.8%	+121.7	CALL	78.7%
Overpair (KK)	83.7%	+117.4	CALL	75.6%
Nut flush draw	72.8%	+95.6	CALL	59.2%
Top pair, weak kicker	85.4%	+120.9	CALL	78.2%
Gutshot draw	40.6%	+31.1	CALL	10.8%
Air (72o)	22.1%	-5.7	FOLD	0.0%

Note the gutshot draw: at 40.6% raw equity it looks strong at first glance, but this is a single-street snapshot -- the module prices the immediate call, not the full multi-street decision tree, which is a deliberate scope limitation discussed below.

2. Bankroll Sizing (Kelly Criterion)

Once a decision is +EV, the Kelly Criterion converts the edge into an optimal stake size as a fraction of bankroll, maximizing long-run geometric growth without risking ruin. Full Kelly is theoretically growth-optimal but highly volatile in small samples; the module also supports fractional Kelly (e.g. half-Kelly) for lower variance at a modest cost to long-run growth, a trade-off widely used in both poker bankroll management and position sizing in trading.



Optimal Kelly stake as equity varies, for a fixed pot-size bet. Below 50% equity (the break-even threshold for this bet size) the optimal stake is exactly zero -- Kelly correctly refuses -EV bets rather than sizing them down. A real bug was found and fixed during development here: an earlier version double-counted the returned stake in the payoff calculation, shifting the break-even point to 33% instead of the correct 50% -- caught precisely because the chart didn't match the known theory.

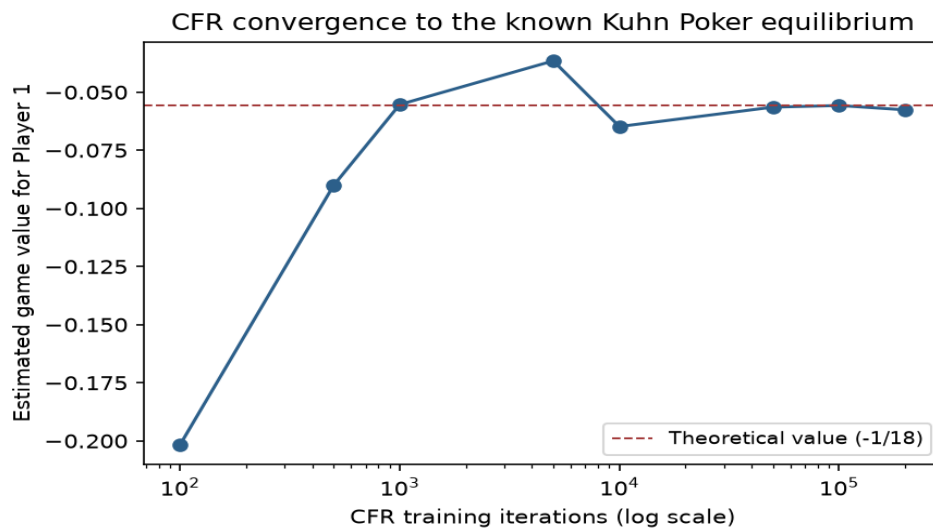
3. GTO Solving via Counterfactual Regret Minimization

Kuhn Poker (a simplified 3-card game: Jack, Queen, King, one betting round) has a known, provably optimal equilibrium strategy, famous for the result that a rational player should bluff the worst hand (the Jack) with probability in $[0, 1/3]$. Rather than hard-coding this answer, the solver learns it from 200,000 iterations of self-play using Counterfactual Regret Minimization (CFR), the same self-play logic used by modern superhuman poker AI (e.g. Libratus, Pluribus), applied here at a small enough scale to solve exactly and verify against the textbook result.

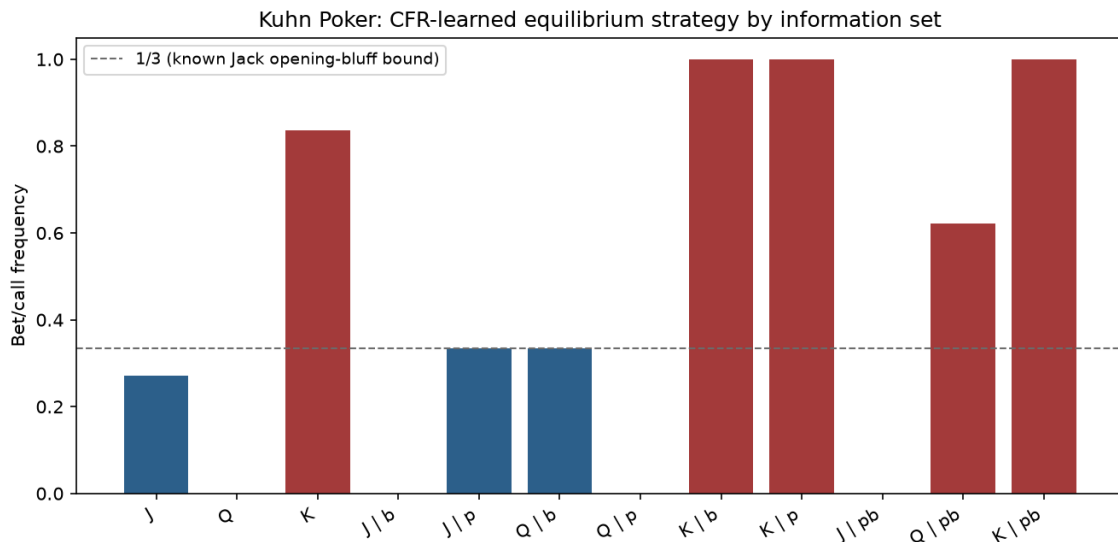
How CFR works, step by step:

- 1 Play the game against itself, tracking every decision point (an 'information set': a player's own card plus the betting history so far).
- 2 At each information set, maintain a running 'regret' for every action -- how much better that action would have performed versus the strategy actually played.
- 3 Choose actions in proportion to their positive regret ('regret matching'): actions that would have done better get played more often in future iterations.
- 4 Track a running average of the strategy played at each information set over all iterations -- this average, not the strategy at any single iteration, is what converges to the Nash equilibrium.

- 5 Repeat for many iterations; the average strategy's exploitability provably shrinks toward zero as iterations increase.



The solver's estimated game value converges to the known theoretical value of $-1/18$ as training iterations increase, confirming the self-play process is actually finding the equilibrium rather than an arbitrary fixed point.



Learned bet/call frequency at every information set. The model independently recovers the $\sim 1/3$ opening bluff frequency with the Jack, and the theoretically required $1/3$ bet frequency with a Queen after the opponent checks then bets -- neither value was hard-coded; both emerged purely from repeated self-play.

Limitations and Future Work

- The GTO solver is exact only for Kuhn Poker (3 cards, one betting round). Real Texas Hold'em has a vastly larger information-set space; extending the solver would require abstraction techniques (card bucketing, action abstraction) used in production-grade solvers, not brute-force CFR.
- The pricing module evaluates a single decision point in isolation. A full treatment would price the entire remaining game tree (all future streets and bet sizes), the poker equivalent of pricing a path-dependent option rather than a single-period one.
- Equity estimation assumes uniformly random opponent hands. A more realistic model would condition on a plausible opponent range inferred from the betting action so far.
- Kelly sizing here treats each decision independently. A bankroll actually facing a sequence of correlated decisions within one session would need a multi-period Kelly treatment, analogous to dynamic (not static) portfolio optimization.

Appendix: Core Pricing Logic

The two functions below are the heart of the pricing module -- pot odds (the break-even equity threshold) and the fair, zero-EV bet size at a given equity, the poker analogue of a no-arbitrage price.

```
def pot_odds(bet, pot):
    """Required equity to break even on a call."""
    return bet / (pot + bet)

def fair_bet_size(pot, equity):
    """
    Bet size at which calling has exactly zero EV, given equity.
    Solved from: equity*(pot+bet) = (1-equity)*bet
    """
    if equity >= 0.5:
        return float('inf')
    return equity * pot / (1 - 2 * equity)
```

The CFR regret-matching update, the mechanism that lets the GTO solver converge to equilibrium purely through self-play rather than a hand-coded solution:

```
def get_strategy(self, realization_weight):
    strategy = [max(r, 0.0) for r in self.regret_sum]
    total = sum(strategy)
    if total > 0:
        strategy = [s / total for s in strategy]
    else:
        strategy = [1.0 / NUM_ACTIONS] * NUM_ACTIONS
    for a in range(NUM_ACTIONS):
        self.strategy_sum[a] += realization_weight * strategy[a]
    return strategy
```

Summary

All four modules were built, tested against known theoretical benchmarks (AA preflop equity ~85%, Kuhn Poker game value -1/18, classic Kelly break-even points), and connected into a single decision pipeline. One real bug was found and fixed during development using the visualizations themselves as a validation tool, not just a presentation layer. The project's contribution is not any individual algorithm, each of which is well documented in the literature, but the explicit framing of poker decisions through the language of no-arbitrage derivatives pricing: equity as an underlying's price, bets as options to be priced, and bankroll sizing as position sizing under a known edge.